

课程笔记

8000 刀换来的 cc 团队开发经验

jesse-成都团队招人 / 课程笔记

2026 年 4 月 10 日



视频作者/频道: jesse-成都团队招人

发布日期: 2024

视频时长: 约 79 分钟

视频链接: <https://www.bilibili.com/video/BV1ihDtBYEU8/>

目录

1 开篇与整体框架：为什么会有 Team 模式	3
1.1 10 万行零手写项目的工作流概览	3
1.2 单智能体瓶颈：时间、准确率与“死循环 hook”的陷阱	4
1.3 Team 模式实录：多角色并行与无人值守效果	5
1.4 本章小结	7
2 KV Cache 与上下文的秘密：成本控制与性能保障	7
2.1 KV Cache 是什么，为什么决定你的账单	7
2.2 Cloud Code 上下文拆解：200K 去哪了	9
2.3 精简上下文：MCP、Skills 与 cloud.md 的最佳实践	10
2.4 本章小结	10
3 Cloud Code 进阶操作：命令、模型与对话管理	11
3.1 模式选择与对话管理	11
3.1.1 执行模式的选择：dangerously 与 auto	11
3.1.2 继续对话 vs 新开对话	11
3.1.3 手动压缩是核心操作	12
3.1.4 版本控制：优先 git，慎用 checkpoint	12
3.2 模型搭配策略：leader 与执行者的分工	12
3.2.1 leader 用理解/规划型，执行用 Sonnet	12
3.2.2 1M 上下文的真实意义	13
3.2.3 切换模型的禁忌	13
3.2.4 卡点的“急救模型”	13
3.3 实用插件与 Skills 配置	14
3.3.1 上下文占比插件：看得见才能控制	14
3.3.2 任务完成通知插件	14
3.3.3 skills 的项目级与用户级配置	14
3.3.4 Task 提示词要简短，引导回文档	14
3.3.5 为什么不建议全自动项目初始化	15
3.4 本章小结	15
4 Harness Engineering：约束、可观测性与文档化	16
4.1 Harness Engineering 核心思想：AI 不是魔术师	16
4.2 可观测性、CLI 优先与日志策略	17
4.3 防止漂移：文档规范 + CI 约束 + 快速反馈部署	19
4.4 本章小结	20
5 CCTeamQuarter 开源项目：从零构建 Team 工作流	20
5.1 开源项目 CCTeamQuarter 的整体定位	21
5.2 Team 启动流程与六角色设计	22
5.3 文档树与热更新导航地图的架构	23
5.4 落地建议：从 Manual 到自动 hook 的演进	25

5.5	本章小结	25
6	总结与延伸	26
6.1	讲师的收尾建议	26
6.2	我们的综合提炼	26
6.3	下一步行动	26

1 开篇与整体框架：为什么会有 Team 模式

1.1 10 万行零手写项目的工作流概览

整个课程的前提是真实的、烧过钱的生产经验：近 10 万行代码的 Web 项目，全程未手写一行代码，从零一直走到服务器部署。为了完成这个项目，讲师投入了约 8000 到 10000 美元的 token 费用，并持续使用两个每月 200 美元的 Cloud Code Max 会员。这个数字不是为了吓人，而是想让你认识到：当 AI 编程进入“大型项目”领域时，它已经从“偶尔写个脚本”的玩具变成了需要系统工程方法论的生产工具。

这

不是炫技，而是规模效应的必然结果。当代码量逼近 10 万行时，单智能体（单 Agent）的即兴发挥模式会迅速失效。你需要的是角色分工、文档约束和可观测性——这些正是本章乃至全书的核心主题。

这个项目复现了 OpenAI 提出的 约束工程 (Harness Engineering) 中约 60%–70% 的流程。讲师本人并非资深工程师，对服务器购买、部署运维等经验非常有限，整个部署链路也是在 Cloud Code 的指导下逐步完成的。这意味着：如果你已经有一定的工程背景，理论上可以取得比这个案例更好的效果。

整个工作流可以概括为五个阶段：

- **需求对齐与团队启动**：读取项目文档和预定义的角色配置文件，初始化 Team 模式；
- **并行调研**：由多个研究者（Researcher）子智能体并行阅读参考项目源码、梳理实现路径；
- **前后端实现**：前端、后端子智能体根据调研结果分头编码；
- **自动 Review 与测试**：代码审查角色和端到端测试（E2E Test）角色介入，验证实现质量；
- **部署与收尾**：自动推送到服务器，管家整理文档与 CI 状态。

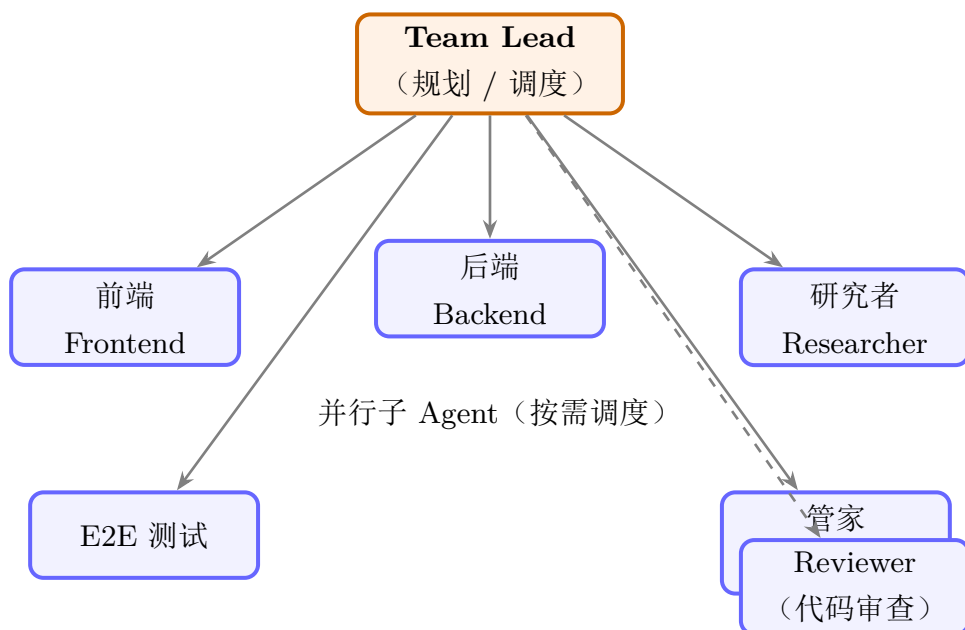


图 1: Team Mode 六角色通信关系示意图。Leader 处于中心，与各子智能体保持并行通信。

整个流程中，人类只负责“给任务”和“在关键节点做 yes/no 决策”，剩余的工作拆分、并行调度、错误排查、文档归类，全部交给 Team 模式自主完成。

1.2 单智能体瓶颈：时间、准确率与“死循环 hook”的陷阱

在引入 Team 模式之前，很多人都会尝试一种“土办法”：用一个单智能体跑，出错或者超时了就通过 hook（死循环）把它拉回来，再让它读文档继续。这个方案有两个致命的隐藏成本。

第一，准确率随时间快速衰减。根据已有测试数据，以 Claude Opus（4.5 / 4.6）为例：

- 持续运行 1 小时：成功率约 90% 及以上；
- 持续运行 2 小时：成功率降至约 80%；
- 持续运行 5 小时：成功率跌至 50% 左右。

⁰视频画面时间区间：00:08:00-00:12:00。若为生成图，则基于视频内容由 TikZ 绘制。

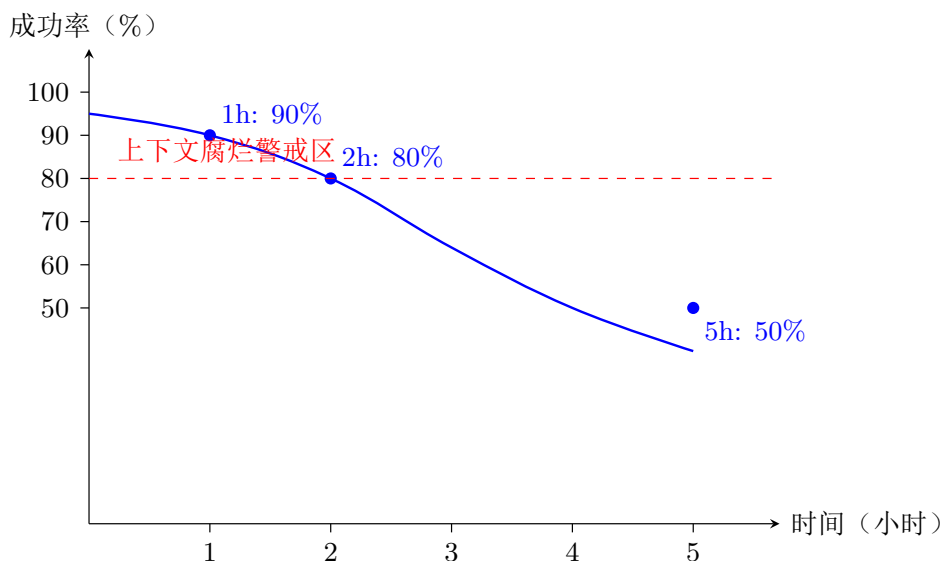


图 2: 单智能体成功率随运行时间衰减的示意曲线。数据来源于讲师在 Claude Opus 上的实测观察。

这意味着，如果你期望一个单 Agent 无人值守地跑上半天，那么它有一半概率会跑偏。而在大型项目中，“跑偏”往往不是直接报错，而是默默引入一个兜底的错误实现、乱改已有接口，或者陷入局部最优的死循环。

局部最优与上下文腐烂

局部最优 (Local Optimum) 指模型在修复某个小错误时，为了通过当前测试而引入一个临时补丁，却逐渐破坏整体架构。上下文腐烂 (Context Rot) 则指随着对话长度增加，模型对早期约束、接口约定和架构决策的记忆逐渐模糊，导致后续修改与最初设计背道而驰。

第二，hook 模式缺乏角色分工和文档规范。死循环 hook 本质上完全依赖模型自身的能力——模型既要当架构师，又要当编码员，还要当测试员和文档整理员。稍微复杂的问题一旦陷入局部最优，它就像掉进坑里，没有太多外部机制能把它拉出来。因此，讲师明确不建议这种方式：

不

要依赖“单智能体 + 死循环 hook”做长时间任务。它既无角色隔离，也缺乏约束边界。除非你只想让模型重复执行某个简单循环，否则性价比最高的做法是一开始就引入 Team 模式。

真要跑长任务，Team 模式几乎是必由之路。在 Cloud Code 的 Team 模式中，多个子智能体 (Sub-agent / Member) 被创建出来，各自有明确的角色边界；leader 负责规划和派发任务，研究者负责并行调研，前后端负责实现，reviewer 和 E2E 测试负责把关。这种分工让单个 Agent 的上下文负担大幅降低，从而显著延长有效工作时间。

1.3 Team 模式实录：多角色并行与无人值守效果

Team 模式的核心价值可以用一句话概括：用角色隔离换上下文清爽，用并行调研换时间效率。

⁰基于视频口述数据由 TikZ 绘制。

讲师展示了一个典型的 Team 模式运行实录。进入 Cloud Code 后，首先让它读取项目文档和 CCTeamQuarter 的预配置文件，随后自动创建团队。典型的角色配置包括：

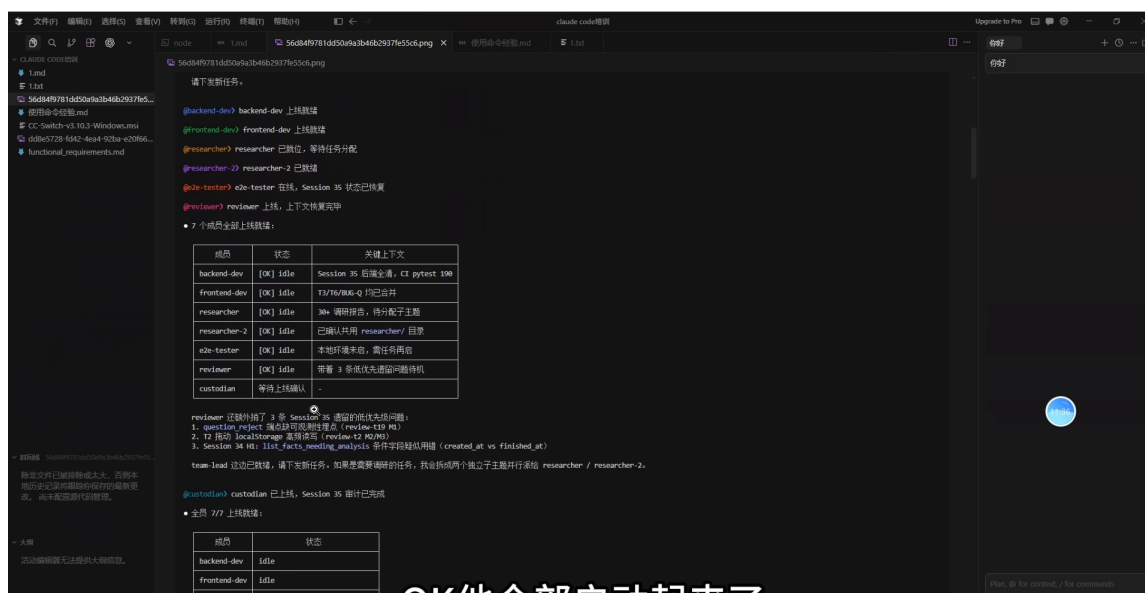


图 3: Cloud Code Team 模式角色总览表格。

- **Leader**：主对话框，负责理解需求、拆分任务、调度成员；
- **后端开发**：处理 API、数据库、业务逻辑；
- **前端开发**：处理 UI、组件、页面交互；
- **研究者 × 2**：并行阅读参考源码、调研实现方案；
- **端到端测试**：使用 Playwright（通过 MCP 或 skills）跑完整链路测试；
- **代码审查**：对实现代码做静态检查与风格把控；
- **管家**：维护任务级三文档（plan、findings、progress），并检查 CI 与文档归类。

下面是一段从实录中提炼出的典型交互流程：

1. 人类一次性抛出 4-5 个彼此独立或弱相关的任务指令；
2. Leader 自动将它们拆分为若干组，并行分发给研究者调研；
3. 研究者返回调研结果后，leader 再派发给前后端实现；
4. 实现完成后，自动触发代码 review，随后进入 E2E 测试；
5. 测试通过，管家检查文档和 CI 状态，整理遗留问题。

在这个过程中，人类只需要在几个关键决策点做回应，比如“这个问题先搁置，后面再说”“你们继续执行”“提醒一下端到端测试还没用”等。其余时间可以完全离开，让 Team 模式自主运行数小时。

⁰ 视频画面时间区间：00:08:30-00:10:00。

大

模型之间的“逻辑能力”差距，没有你想象的那么大。真正拉开差距的是注意力：好的 leader 模型（如 GPT-5.4、Claude Opus、Kimi K2.5）能在长对话中仍然记得“先调研、再实现、再 review、再测试、最后管家检查文档”的完整流程；而注意力较差的模型，可能在某一步就把前面的指令忘记了。

这也是讲师推荐 leader 使用具备 1M 上下文能力模型的原因——它的真正价值在于“压缩自主权”，详情见第 3 章。

1.4 本章小结

本章围绕“为什么必须有 Team 模式”展开了三层论证：

- **规模层面**：当项目达到近 10 万行代码时，单智能体的即兴模式必然失效；
- **机制层面**：单智能体存在准确率随时间衰减、局部最优、上下文腐烂等系统性瓶颈，死循环 hook 无法解决；
- **实践层面**：Team 模式通过 leader + 多子智能体的角色隔离，实现了任务拆分、并行调研、自动 review 与无人值守部署，是把 AI 编程从“玩具”推进到“生产工具”的关键一跃。

下一章，我们将深入 KV Cache 的命中机制与 Cloud Code 的上下文构成，理解它们如何决定你的账单与任务上限。

2 KV Cache 与上下文的秘密：成本控制与性能保障

在大量使用 Cloud Code 进行长对话开发时，很多人会惊讶地发现：月初余额消耗得飞快，或者模型明明标称 200K 上下文，usable 空间却只有一百多 K。这一节我们先从账单的根源——KV Cache 的命中机制——说起，再解剖 Cloud Code 的上下文到底被哪些东西占满，最后给出经过实战验证的精简策略。

2.1 KV Cache 是什么，为什么决定你的账单

直觉上，大模型每一次回复，都是在“读完全文之后写下一个字”。如果你已经聊了很久，模型是不是每次都要把之前成千上万 token 全部重新读一遍？如果没有优化，成本将高到无法承受。

幸运的是，Transformer 的自回归推理存在一个关键性质：对于已经处理过的历史 token，它们的 Key (K) 和 Value (V) 矩阵是固定不变的。于是我们可以把这些 K 、 V 存下来，下次只计算新 token 的 Query (Q)，再与缓存的 K 、 V 做注意力计算即可。这种复用机制就是 **上下文缓存 (KV Cache)**。



图 4: KV Cache 原理示意图：新 token 仅计算 Query，并与缓存的历史 Key/Value 拼接完成 Attention 计算。

以一个简化公式为例，第 t 步的注意力输出可以写成：

$$\text{Attention}_t = \text{softmax} \left(\frac{Q_t [K_1, K_2, \dots, K_t]^T}{\sqrt{d_k}} \right) [V_1, V_2, \dots, V_t]$$

其中：

- Q_t ：当前新 token 的 Query 向量；
- $[K_1, \dots, K_t]$ ：从第 1 个 token 到当前 token 的所有 Key 向量拼接；
- $[V_1, \dots, V_t]$ ：对应的所有 Value 向量拼接；
- d_k ：Key 向量的维度，用于缩放。

在这个过程中， $K_1 \dots K_{t-1}$ 和 $V_1 \dots V_{t-1}$ 都来自缓存，只有 Q_t 和新产生的 K_t, V_t 需要实时计算。如果缓存失效，前面全部 K/V 都得重算，成本会呈指数级放大。

没有 KV Cache 命中率的中转站推荐，就是在耍流氓。讲师的原话虽尖锐，但道理不虚：一次大请求若输入输出合计 380K token，在没有缓存的场景下，账单会是命中缓存场景下的近十倍。很多号称“送两亿 token”的平台不提缓存，是因为这两亿在实际无缓存的高频调用中可能几小时就烧光。

千

万不要在对话中途修改历史消息。Cloud Code 允许你回头编辑某条历史记录，但只要你改动了中间任意一条（例如把某条消息里的时间更新为当前动态时间），从那一條往后全部的 KV Cache 都会报废。前面部分还能复用，后面全部需要重新计算，这个成本是你完全无法接受的。同理，**不要在完成一个任务的过程中切换模型**——切换一次，缓存即清零。

⁰ 视频画面时间区间：00:15:45–00:17:10。

还有一个常见误区：先用便宜模型“跑上下文”，再换好模型做精细推理，是不是更省？答案是否定的。就算便宜模型的调用费用再低，也低不过已有 KV Cache 完全命中时的边际成本。对于 Cloud Code 这种工具调用频繁、上下文极长的场景，命中缓存带来的收益远大于换用廉价模型的节省。

2.2 Cloud Code 上下文拆解：200K 去哪了

很多人看到 Cloud Code 提供 200K 上下文，第一反应是“够放很多代码了”。但真实可用空间往往只有不到 60%。下表总结了讲师优化后的上下文占用构成（以 200K 总长度为例）：

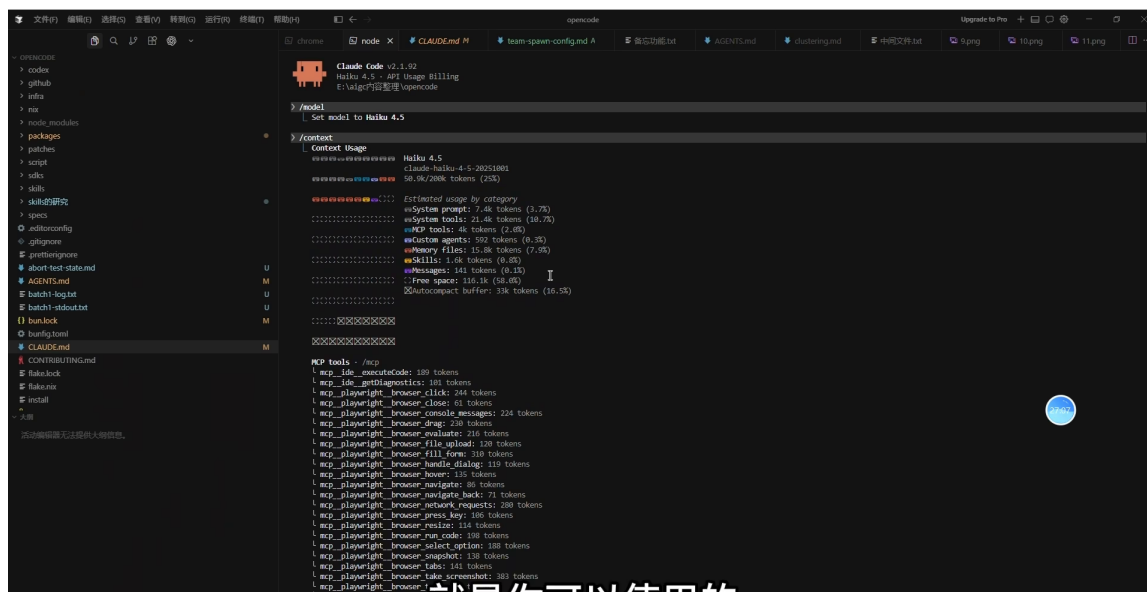


图 5: Cloud Code 上下文占比插件界面，展示 200K 上下文中各模块的 Token 消耗分布。

- **自动压缩预留**：约占 16%（约 33K）。这是系统为“上下文压缩”预留的缓冲空间。实际压缩结果通常只有 3K–4K，因此这个余量显得过大，但用户无法控制。
- **系统提示词 + 系统工具**：约占 30K，完全不可删减。
- **MCP 工具**：这是最大的用户可控黑洞。讲师只装了一个 Playwright MCP，就已消耗可观 token；之前装多个 MCP 时，可用空间直接跌到 80K。
- **用户 Agent（子智能体配置）**：定制的子 agent 描述，token 不多，但装大量插件时可能附带额外子 agent。
- **记忆文件（cloud.md 等）**：讲师的 cloud.md 约两三百行，占了 5.5K token。优化后仍建议控制在 100 多行。
- **Skills（技能）**：只加载元数据，每个 skill 占用很少，可相对放心地多装几个。
- **历史消息（Message）**：剩下的才是真正留给对话历史与代码引用的“自由空间”。优化后约为 110K；未优化时仅 80K 左右。

⁰ 视频画面时间区间：00:20:30–00:22:00。

即

使“什么都没装”，Cloud Code 的可用上下文通常也只有 120K–130K。只要你装上必要的 MCP、skills 和记忆文件，可用量会进一步下滑。因此在进入长对话之前，先打开上下文占比插件检查一次，做到心中有数。

关于记忆文件还有一个细节：member（成员记忆）会自动记录你的习惯与规则，但系统只会加载前 200 行，并且以简要规则为主。你可以定期审视并优化这些记忆文件，但要注意——过度精简会导致行为偏好丢失，得不偿失。

2.3 精简上下文：MCP、Skills 与 cloud.md 的最佳实践

既然上下文是稀缺资源，我们就要像管理内存一样管理它。以下是讲师经过大量试错后提炼出的三条核心原则。

1. MCP：能少装就少装，优先转为 Skills 按需加载

MCP 工具的最大问题是“全量常驻”。除非某个工具是高频必需且需要模型时刻知道它的存在，否则应尽量把它做成 Skill。Skills 是按需动态加载的，初始 token 消耗远低于 MCP。如果你不是每天都要用到某个 MCP，那就卸载它，改为 Skill 调用。

2. Skills：控制在 2–4 个为宜

虽然 Skills 的单个元数据很小，但装多了会分散模型注意力。根据社区 benchmark，2–4 个 skill 时触发最精准；超过 12 个后，模型不仅容易“笨”，还可能该触发的时候不触发。如果你需要强制调用某个 skill，建议在 prompt 里显式点名。

3. cloud.md：最好控制在 100 多行

cloud.md 不是越详细越好。它的最佳内容应聚焦于：

- 项目文件结构概览；
- 核心规划与规则（应该怎么写、怎么组织）；
- 反复出现的失败模式与行为偏好（例如“优先使用某库”“禁止某种写法”）。

详细的 API 文档、长篇设计说明不该塞在这里，而应放在项目文档树中，由 agent 按需读取。

避

免在对话中途切换模型或修改历史消息。这不仅会打断 KV Cache，还可能让模型在状态切换后遗忘早期约束。完成一个复杂任务时，最好“从上到下”一路到底；如果真需要换模型，建议先结束当前对话，再在新对话中重新交代背景。

2.4 本章小结

本章的核心要点可以归纳为三句话：

- **KV Cache 决定账单厚度**。在自回归推理中，历史 K/V 的复用能把长对话成本降低一个数量级；任何破坏中间键的行为（改历史、切模型）都会让后面全部缓存报废。
- **200K 上下文并不可全用**。系统预留 + MCP + 记忆文件会吃掉近一半空间；优化后可用的自由上下文大约 110K，未优化时可能只剩 80K。

- **精简策略：**卸载非必要 MCP、控制 Skills 在 2-4 个、cloud.md 保持 100 多行。把大段文档从常驻上下文移出，转为按需读取，是保障长对话稳定性与成本可控的关键。

掌握了上下文原理后，下一章将进入 Cloud Code 的操作层细节——如何通过模式选择、对话管理与模型搭配来进一步保障长对话的稳定性。

3 Cloud Code 进阶操作：命令、模型与对话管理

上一章我们搞清了 KV Cache 和上下文的底层原理，知道了“钱花在哪”。本章聚焦于 Cloud Code 中被频繁低估的“操作层”细节——模式怎么选、模型怎么搭、对话怎么管、插件怎么配。这些看似琐碎的决策，叠加起来就是 10 万行项目能否稳定推进的底层基础设施。正如开车时如果你不懂得换挡和保养，再贵的引擎也会早衰。下面将这些决定成败的实操经验逐一展开。

3.1 模式选择与对话管理

3.1.1 执行模式的选择：dangerously 与 auto

Cloud Code 在执行外部命令时会提供权限控制，核心是两个档位：

- **dangerously 模式：**模型可以直接运行终端命令、读写文件，无需逐条人工确认。
- **auto 模式**（较新版本推出）：系统会尝试自动判断哪些命令需要用户批准。

讲师的个人经验是，只要项目不是高度机密、用户数据极敏感或风险容忍度极低，**优先开启 dangerously 模式**。原因很直观：如果你因为担心潜在风险而给自己设了层层防线——比如每次命令都要点同意——那么频繁的打断会让长对话的“心流”彻底断裂，模型也更容易在停顿后偏离原有方向。

风险与效率的权衡

真正因模型误操作导致“删库跑路”的案例在常规开发中极为罕见。为了极低概率的风险，牺牲高频次的流畅体验，是得不偿失的。如果你的风险敏感度在可接受范围内，建议大胆放开执行权限，把精力放在约束文档和版本控制（git）上。

讲师的原话很直白：“只要你把 Harness 约束、规范弄好，多提醒它，哪怕模型弱一点也能完成任务。”执行权限只是辅助，真正的防线在文档和版本控制。

3.1.2 继续对话 vs 新开对话

一个常见的疑问是：当前任务没做完，下次应该“继续之前的对话”，还是重新开一个？讲师的观点很明确：

对话管理原则

最好不要继续旧对话。要么在关键节点主动压缩上下文并收尾，要么直接结束会话、下次新开。真正该保留的上下文，应该通过文件（如 plan 文档、cloud.md、进度摘要）沉淀到工作目录中，而不是挂在会话历史里。

在讲师的 CCTeamQuarter 工作流中, Team 模式会把所有关键内容写进工作目录下的 `project-docs/` 文件夹。下一次启动时, 只要把这些文件一起发给模型, 它就能自行恢复上下文继续执行。这种方式的好处是:

- 避免了旧对话中无关紧要的调试记录、失败尝试继续占用 KV Cache;
- 人为控制“重新开始”的节奏, 相当于给模型一次清空杂念的机会;
- 文件沉淀的内容可以随时被人类审阅、修改和版本控制。

3.1.3 手动压缩是核心操作

Cloud Code 支持在对话过长时自动压缩历史消息, 但讲师反复强调:

自动压缩的隐患

压缩必须由你自己选择时机, 而不是让系统自动压缩。这一点值得再三强调。自动压缩时, 系统可能会丢弃你尚未文档化的关键约束、任务变更或失败经验, 导致后续修复循环。

推荐的压缩时机包括:

- 一个大任务或一整个功能模块完成之后;
- 上下文接近窗口警戒线 (如 200K 或 300K) 之前;
- 对话开始出现“上下文腐烂”迹象——模型反复问同样的问题、开始遗忘早期约束时。

在 1M 上下文模型中, 获得的最大价值并非真的可以塞满 1M token, 而是**你可以自主选择压缩的时机**, 而不必在 200K 或 275K 时被系统强行截断。

```
1 我准备压缩上下文了。请在这个任务上做一些收尾工作:
2 1. 检查还有哪些未完成的子任务或测试;
3 2. 将关键发现更新到 plan.md 和 findings.md;
4 3. 确认文档归类完毕后再结束。
```

Listing 1: 压缩前向模型发出的收尾指令示例

3.1.4 版本控制: 优先 git, 慎用 checkpoint

Cloud Code 近期新增了内置 checkpoint (对话级版本回退) 功能, 可以双击消息消除或按 `Ctrl+U` 删除单行消息。但讲师的建议是:

版本控制策略

尽量使用 git 做版本控制, 而不要把 checkpoint 当成主要防线。内置版本回退有时表现不稳定——某些版本可以正常恢复, 某些版本又会带入新的状态错误。相比之下, git 的提交记录更清晰、可预期, 也便于多个子智能体或人类开发者协同。

3.2 模型搭配策略: leader 与执行者的分工

3.2.1 leader 用理解/规划型, 执行用 Sonnet

并不是所有任务都需要最强模型。合理的分工可以显著降低 token 成本, 同时保证质量:

- **leader / 规划角色**：负责需求理解、任务拆解、子智能体调度。推荐使用理解能力强、指令遵循稳定的大模型，如 Claude Opus、GPT 5.4 或 Qwen 3.6 (1M 上下文)。
- **机械实现角色**：负责写具体函数、页面、测试脚本。这类工作上下文相对聚焦，用 Sonnet 足矣，性价比更高。

讲师特别提到，GPT 5.4 在步骤规划、并行处理和指令遵循上表现优异，甚至比 Opus 更稳定。Opus 虽然推理深度强，但在长对话中偶尔会遗忘细节；而 GPT 5.4 能更可靠地记住 Team 模式的完整流程：先做研究，再交给前后端实现，再做代码审查，最后跑测试和部署。

3.2.2 1M 上下文的真实意义

很多人误以为 1M 上下文意味着可以把整个代码库塞进去让模型一次性处理。实际上：

关于长上下文的误解

1M 上下文的核心价值不在于“用到 1M”，而在于“由你决定何时压缩”。模型在 300K 之后就会出现明显的上下文腐烂：早期任务与当前任务相关性低，导致模型注意力分散、容易陷入局部最优。因此即便有 1M 窗口，讲师也只在极少数情况下用到 500K，并会在 300K 左右主动收尾准备压缩。

对于 Team 模式的 leader 来说，它本身不处理大量具体代码，而是派发任务、维护文档和协调成员。如果 leader 模型只有 275K 上下文，可能在多个并行任务尚未完成时就被系统强制压缩，从而丢失关键进度信息。因此，**为 leader 配备 1M 上下文模型是 Team 模式稳定运行的基础设施。**

3.2.3 切换模型的禁忌

在同一对话中途切换模型，是极其昂贵且低效的操作。原因在于：

中途换模型

当你在一个已有长上下文的对话中切换模型时，前面的 KV Cache 会被直接破坏，后续推理需要重新计算几乎全部历史 token。成本之高，往往超过使用贵模型本身的差价。

只有在以下边界点才适合换模型：

- 手动压缩上下文之后；
- 新开一个对话会话时；
- 对话已经结束，下一轮任务重新开始时。

3.2.4 卡点的“急救模型”

当某个难题 Sonnet 反复陷入死循环、或者出现局部最优爬不出来时，可以临时调用“高推理档”模型作为急救：

- **Codex 5.4** 的最高推理性能模式；
- **Claude Opus** 将 thinking 档位调到最高。

对于绝大多数 Web 项目而言，问题往往不是模型“解不了”，而是模型“在已有文档和上下文干扰下解偏了”。此时也可以尝试让模型忽略之前的失败记录和相关文档，靠自己重新推理，这往往能打破局部最优。

3.3 实用插件与 Skills 配置

3.3.1 上下文占比插件：看得见才能控制

讲师强烈推荐一款名为 **Cloud Hold** 的插件。它可以在界面侧边栏实时显示：

- 当前对话已消耗的上下文 token 量及占比；
- 模型调用了哪些工具；
- 模型读取了哪些文件。

这个信息对于决定“何时压缩”至关重要。另一个相关功能是，在 Team 模式下按 **Shift + ↑ / ↓** 可以快速切换不同的 team 成员（子智能体），方便你查看各自的状态。

上下文可视化的价值

只有当你能直观看到上下文消耗时，才能精准控制压缩时机。不要凭感觉判断对话“好像还能继续”——插件数据不会让你误判。

3.3.2 任务完成通知插件

在并行开发多个任务时，人类很容易遗忘自己开了哪些 Cloud Code 对话。可以安装任务完成通知插件（如 `cloud-done-notification` 类扩展），当某个子智能体或对话完成时，右下角弹出提示。讲师使用 Team 模式后对这一需求有所降低，因为大部分并行任务都在同一个 Team 会话中由 leader 统一调度，只需监控一个主对话即可。

3.3.3 skills 的项目级与用户级配置

Cursor / Cloud Code 的 **skills** 支持两级配置：

- **项目级**：放在当前工作目录下，只对当前项目生效；
- **用户级**：全局可用，所有项目都能加载。

skills 的触发机制是按需加载元数据，初始 token 占用远低于 MCP。因此 skills 可以相对灵活地配置，但也不宜过多。根据实测经验，**同时加载 2–4 个 skills 效果最佳**；超过 12 个时，模型的触发精准度和注意力分配都会下降。如果你在某些场景下必须依赖特定 MCP 工具，也可以将其封装为 skill，实现按需加载，进一步节省初始上下文。

3.3.4 Task 提示词要简短，引导回文档

Cloud Code 现在支持 **task** 机制：模型会将任务写成 markdown 文件，存放在自己的工作目录下，然后按文件指令执行。这比过去纯对话式的 to-do list 更有文件沉淀，但也有一个问题——这些 task 文件默认不在你的项目文件夹里。

讲师的处理方式是：

- 让 task 文件只写**聚焦的核心摘要**，不要写冗长详细的实现步骤；
- 在 task 中引用项目内具体的 **plan 文档**，把详细内容引导回你可以控制的文件；
- 确保自定义提示词不要与 Cloud Code 内置的系统提示词冲突，以免造成模型困惑。

```

1  ## 当前任务：实现用户登录 API
2
3  - 按 backend-plan.md 中的接口规范实现 /api/login
4  - 使用已有的 db/schema.ts 中的 User 表结构
5  - 完成后将测试用例写入 tests/auth.test.ts
6  - 详细规范见：docs/plan/backend-plan.md

```

Listing 2: 推荐的 task 文件写法示例

3.3.5 为什么不建议全自动项目初始化

Cloud Code 或某些插件提供了“让模型全自动扫描项目并生成 cloud.md”的功能。讲师对此持明确反对态度：

cloud.md 的初衷与陷阱

cloud.md 的设计目的是给模型提供一张项目“地图”，让它快速定位架构和文件。但实测表明：

- 自动生成的大型地图**并不能提高准确率**；
- 反而会让模型因为“有地图”而被迫查看更多不必要的细节，导致 token 消耗增加 20% 以上；
- 用 grep / 正则 / 关键词匹配的方式，已经足以解决大部分文件定位问题。

cloud.md 更适合用来记录**踩坑经验、特殊规则、工作流约定、子智能体角色清单**——这些是经由你提炼后的“教训”和“偏好”，而非机器自动生成的目录树。长度上，**最好控制在 100 多行**，最多不超过两三百行。

3.4 本章小结

本章围绕 Cloud Code 的“操作层”展开了系统讲解，核心要点可归纳为以下四条：

1. **模式与对话管理**：常规开发优先用 dangerously 模式释放效率；不要恋战旧对话，该压缩时主动压缩，把上下文迁移到文件沉淀中；永远优先用 git 做版本控制。
2. **上下文压缩是可控艺术**：1M 上下文的最大价值是“你决定何时压缩”。在 200K-300K 区间主动收尾、保留文档，是防止上下文腐烂和局部最优的关键。
3. **模型分工与切换纪律**：leader 配强规划型模型（Opus / GPT 5.4 / 1M 上下文），执行者用 Sonnet；中途切勿换模型，急救只在压缩后或新对话中使用。
4. **插件与 skills 配置**：用 Cloud Hold 等插件监控 token 消耗；skills 2-4 个为佳，MCP 尽量按需封装为 skill；task 提示词要简短并引导回项目文档；cloud.md 记录经验教训而非自动生成架构地图。

这些看似琐碎的“小技巧”，叠加起来就是 10 万行项目能否稳定推进的底层基础设施。下一章我们将进入更上层的工程约束方法论——Harness Engineering。

4 Harness Engineering：约束、可观测性与文档化

在团队模式里，我们把一个复杂项目拆成多个子任务并行推进。但拆得再细，如果缺少“缰绳”，智能体就会像脱缰的野马一样越跑越偏：API 端口偷偷改、变量名随意编、上下文一压缩就失忆。OpenAI 提出的 **Harness Engineering**（约束工程），本质上就是给这匹野马套上三根缰绳——文档、测试与可观测性。本章将结合讲师的实战经验，把这套方法论拆成可落地的动作清单。

4.1 Harness Engineering 核心思想：AI 不是魔术师

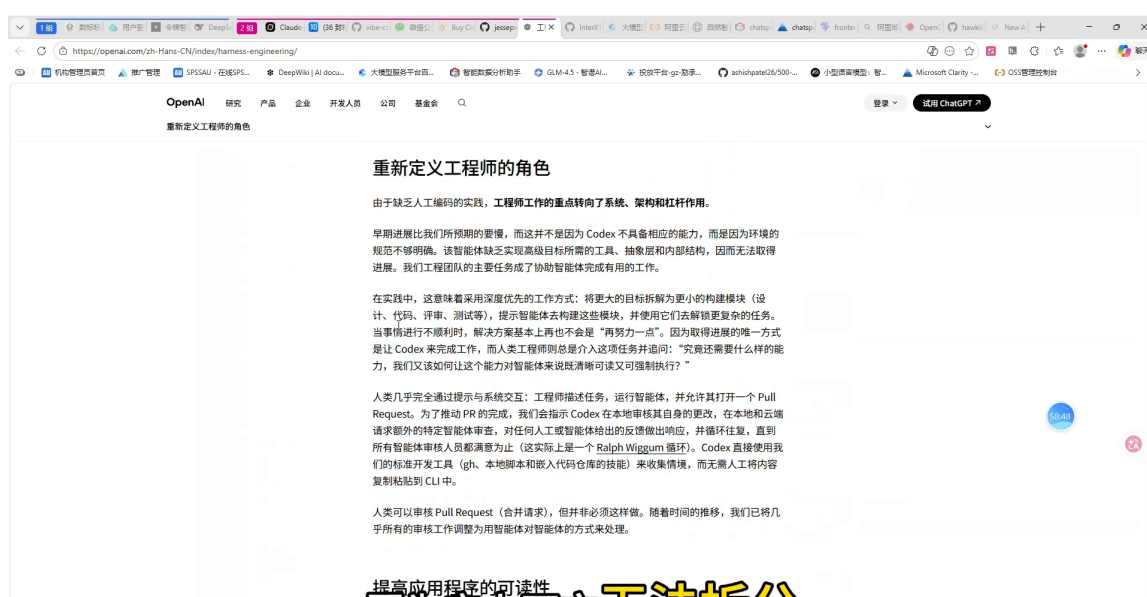


图 6: OpenAI 博客《Harness Engineering》文章页面，系统阐述约束工程核心理念。

OpenAI 内部有一个团队，用 GPT-5 和 Codex 在 5 个月内从零写出一百万行代码，平均每天处理 3.5 个 PR，工程师没有手写一行业务代码。支撑这一成果的，正是 **Harness Engineering** 这篇工程博客。讲师反复强调：这篇文章值得读很多遍，直到你能把它翻译成自己项目的检查清单。

核心直觉

AI 尚不能“化腐朽为神奇”。它擅长在清晰边界内高效执行，却会在边界模糊时自作聪明。Harness Engineering 的目标，就是把边界用 **文档、测试和可观测性** 钉死。

从”死循环 CLI”到标准开发流程

OpenAI 团队最初的尝试也是一个简单的“死循环”：清工作台 -> 操作应用 -> 看日志 -> 截图 -> 修复 -> 重启，循环往复。这个模式与讲师的项目如出一辙，但光靠循环不够，必须把以下要素标准化：

⁰ 视频画面时间区间：00:44:30–00:46:30。

- **系统提示词 (System Prompt)**: 把角色、边界、失败模式注入到每次对话的起点;
- **代码与测试 CI 配置**: 每次产出都必须经过自动化检查;
- **内部开发者工具**: 如 Playwright 驱动的 E2E Test、浏览器开发者工具抓痕;
- **不断提高的自主水平**: 最终让新需求只需一个 hook 就能自动流转。

为什么不要依赖”一兆上下文”

Cloud Code 支持高达 1M 的上下文窗口, 但讲师和 OpenAI 都警告: 真正重要的不是把 1M 塞满, 而是 **在到达临界点之前, 由人来决定压缩时机**。自动压缩往往发生在模型已经迷失、文档尚未沉淀的时刻; 而手动压缩可以让你先把 plan、findings、progress 写入文件树, 再清空前文, 实现真正的“上下文接力”。

压缩时机的误区

不要在对话中途切模型或依赖自动压缩。应在任务收尾、上下文接近 200K–300K 时, 主动沉淀文档并手动压缩。否则 KV Cache 会被破坏, 模型仿佛“换了一个新员工”, 对前期约定一无所知。

4.2 可观测性、CLI 优先与日志策略

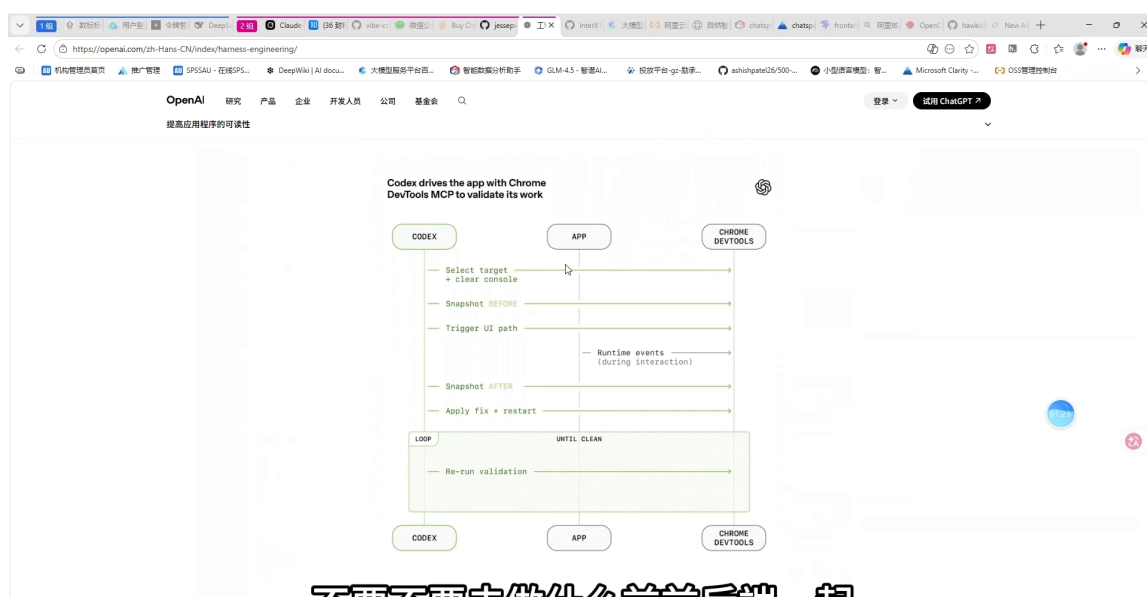


图 7: OpenAI 发布的”Code drives the loop” 流程图, 展示日志、截图、修复之间的标准化循环。

如果只能给 Team 模式添加一个基础设施, 讲师的答案是: **详尽的可观测性**。这不仅是为了人看, 更是为了让 AI 在报错后能快速定位。

CLI 优先: 先让功能跑在命令行里

讲师提到一个常被忽视的优先级:

⁰视频画面时间区间: 00:46:30–00:48:30。

```

1 # 1. 先把核心逻辑暴露为 CLI
2 python my_feature.py --input data.json
3
4 # 2. 通过 CLI 测试边界与异常
5 python my_feature.py --input bad_data.json
6
7 # 3. 最后再打包成前端或桌面应用
8 npm run build:frontend

```

Listing 3: 功能验证的推荐顺序

原因很现实：前后端联调、UI 渲染都是“细枝末节”，而功能内核才是 AI 最容易反复出错的环节。把内核 CLI 化之后，你可以用最小的上下文验证输入输出，再把“不值钱的”UI 慢慢包上去。

日志要足够详细，因为 AI 用正则搜索

OpenAI 的测试循环里同时跑了三套日志系统。对普通开发者来说，不必关心具体是哪三套，只要记住一条原则：

日志策略

日志尽量详细。 AI 不会傻乎乎地从头到尾读完，而是会用类似正则或关键词搜索的方式抓取。因此，日志里要包含命令名、调用参数、模型调用点（提示词注入 / prompt injection 点）、截图前后的状态摘要。生产环境可以关，调试环境必须开到最大。

一个实践示例是，让子智能体在执行每个关键步骤后输出结构化日志：

```

1 {
2   "step": "api_call",
3   "endpoint": "/v1/tasks",
4   "status": 200,
5   "timestamp": "2026-04-10T12:34:56Z",
6   "context_tokens": 184320,
7   "next_action": "call_e2e_agent"
8 }

```

Listing 4: 推荐的可观测性日志片段

当后续排查时，leader 可以直接搜索 "status": 200 或 "context_tokens"，而不必逐行阅读文本。

4.3 防止漂移：文档规范 + CI 约束 + 快速反馈部署

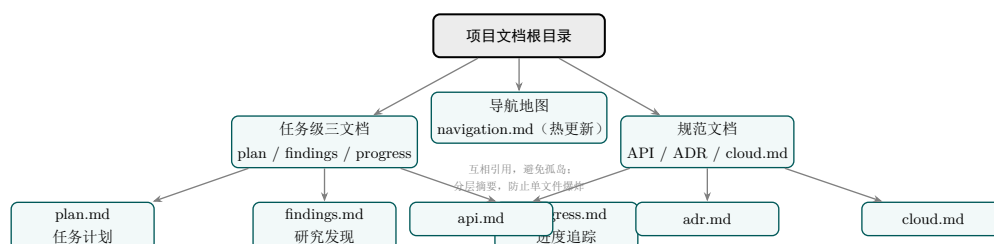


图 8: CCTeamQuarter 文档树结构示意图：采用树状摘要 + 引用细节，避免单文件爆炸与文档孤岛。

AI 在上下文衰减后最常见的“作弊”行为，就是 **乱编一个新变量或新函数**。你明明已经定义了 `user_id`，它却在另一个文件里偷偷引入 `uid`，然后报错 → 修修补补 → 技术债越堆越多。防止这种**代码漂移**，需要三层防线。

第一层：规范文档固定边界

把不能变的东西写进文档，并且写成“死规矩”：

- API 路径与端口不得随意变更；
- 全局变量命名必须在规范列表内；
- 新增第三方库需要说明理由；
- 单个文件不得超过 800 行（或你设定的阈值）。

讲师特别强调：文档不是一本 1000 页的说明书，而是给模型的一张 **地图**。所有都重要等于所有都不重要，因此文档必须采用 **树状摘要 + 引用细节** 的结构：顶层放摘要和索引，底层放具体实现。不能有孤立文档，每个叶子节点都要被某个摘要引用。

第二层：CI 与脚本作为硬约束

提示词是软的，脚本是硬的。讲师建议每个项目在最开始就配置 CI 检查，例如：

```
1  #!/bin/bash
2  # ci/check-harness.sh
3
4  # 检查文件行数
5  find src -name "*.py" | while read f; do
6      lines=$(wc -l < "$f")
7      if [ "$lines" -gt 800 ]; then
8          echo "ERROR: $f exceeds 800 lines ($lines)"
9          exit 1
10     fi
11 done
12
```

⁰基于视频内容由 TikZ 绘制。

```

13 # 检查非法变量名（示例：禁止 uid 替代 user_id）
14 if grep -R "\buid\b" src/; then
15     echo "ERROR: inconsistent variable 'uid' found"
16     exit 1
17 fi

```

Listing 5: 项目约束检查脚本示例

这些脚本在每次子智能体提交后自动运行，能把漂移扼杀在萌芽状态。讲师直言：以前只有大项目才配 CI，现在 Web Coding 项目无论大小都应该从一开始就做。

第三层：快速部署与反馈闭环

AI 时代，合并和部署可以更快、更频繁。你不需要追求尽善尽美，而是要追求 **错误尽快暴露、尽快回流**。讲师分享了一个低成本方案：用户对话结束后，fork 一个轻量级小模型分支，把聊天记录丢给它做快速分析，问它“刚才的沟通里有没有不好的点、skills 要不要更新”。把分析结果自动沉淀为 findings，第二天 review 即可。

部署理念的转变

从“上线前 exhaustive 检查”转向“上线后快速回滚 + 自动反馈收集”。让一个小模型或轻量 agent 长期存活在你的项目里，持续分析用户行为与日志，是保持系统不漂移的秘密武器。

4.4 本章小结

- **Harness Engineering** 的核心是用文档、测试、可观测性给 AI 划定清晰边界，而不是指望它自动超越边界。
- **CLI 优先**能最大化验证效率，把“不值钱的”UI 封装放到最后；日志必须详尽，因为 AI 用搜索而非通读。
- 防止 **代码漂移**需要三层防线：规范文档（树状索引、无孤立文档）、CI 脚本（硬约束）、快速反馈部署（让小模型长期存活并分析）。
- 压缩上下文时，务必由人主动收尾并沉淀文档，避免自动压缩导致 KV Cache 失效与约束丢失。

方法论有了，下一章将介绍一套可直接落地的开源模板——CCTeamQuarter，教你如何把上述思想搬进真实项目。

5 CCTeamQuarter 开源项目：从零构建 Team workflow

经过前面四章的铺垫，我们已经理解了为什么单智能体会遇到天花板（第 1 章），也掌握了上下文缓存（第 2 章）、Cloud Code 操作技巧（第 3 章）以及约束工程（第 4 章）的核心思想。现在的问题只剩下一个：有没有一套可落地、可迁移的模板，能把这些方法论直接搬进你的项目里？

本章要介绍的就是讲师开源的 CCTeamQuarter（简称 CCTM/QUOR）项目。它的目标很简单——为 Cloud Code 的**团队模式**提供一套开箱即用的角色配置、文档结构和启动流程。你不需要从零去思考该拆几个子智能体、该写哪些计划文档，只需要在它的模板上根据业务微调即可。

5.2 Team 启动流程与六角色设计

CCTeamQuarter 的启动流程可以概括为五步，每一步都有明确的目的：压缩 token 浪费、恢复历史上下文、对齐目标。

1. 读取参考文件（skills、模板）到主 Agent 上下文
2. 检查项目根目录是否已有 plan 文档
 - 有 → 直接读取，复用已有 team 配置
 - 无 → 创建新的计划文档与角色定义
3. 扫描代码库，收集项目上下文
4. 与用户对齐目标、介绍 team 成员与分工
5. 生成 cloud.md，串行创建所有子智能体

Listing 6: CCTeamQuarter 启动流程概要

其中第 2 步的“复用”设计非常关键。如果你之前已经用 CCTeamQuarter 创建过一次 team，项目里会保留一份角色配置文件。下次启动时，系统可以直接读取这份快照，而不需要重新解析所有 skill 细节，从而节省大量 token。

首次创建的上下文开销

如果是全新项目，没有历史 team 配置，那么仅读取 skill 和代码库扫描就可能消耗约 100K 的上下文。因此第一次创建完成后，建议主动压缩上下文，把 skill 的冗长说明清理掉，避免它们长期占用你与模型对话的宝贵窗口。

六角色分工

CCTeamQuarter 推荐的 team 成员如下：

- **Leader**：统筹协调，能看到所有群聊消息，负责任务拆分与最终决策。
- **后端开发**：采用 TDD（测试驱动开发）模式，先写测试用例再丰富实现。
- **前端开发**：同样采用 TDD，负责 UI 实现与交互对接。
- **研究者（探索）**：负责技术调研、方案可行性分析与信息检索。
- **代码审查（Review）**：对前后端代码做静态检查与风格把控。
- **E2E 测试**：用 Playwright 等工具跑端到端测试，验证完整用户链路。
- **管家**：维护文档树、热更新导航地图、整理历史归档。

讲师特别指出，**角色不是越多越好**。拆分的判断标准应该是“上下文能否分开、沟通成本是否低”。如果两个智能体的工作需要频繁深度协同，那就应该合并成一个角色；反之，如果只需要互相 @ 一两句话就能交接，就可以拆分。

多次对齐需求的设计

与 Anthropic 和 OpenAI 的 harness 方案不同，CCTeamQuarter 特别强调了**子智能体之间需要多次对齐需求**。Leader 向下派发任务前，执行者会先与 Leader 确认方案；研究者得出结论后，也会先回传给 Leader 审核，再进入开发。这种“先对齐、再动手”的机制能显著降低返工率。

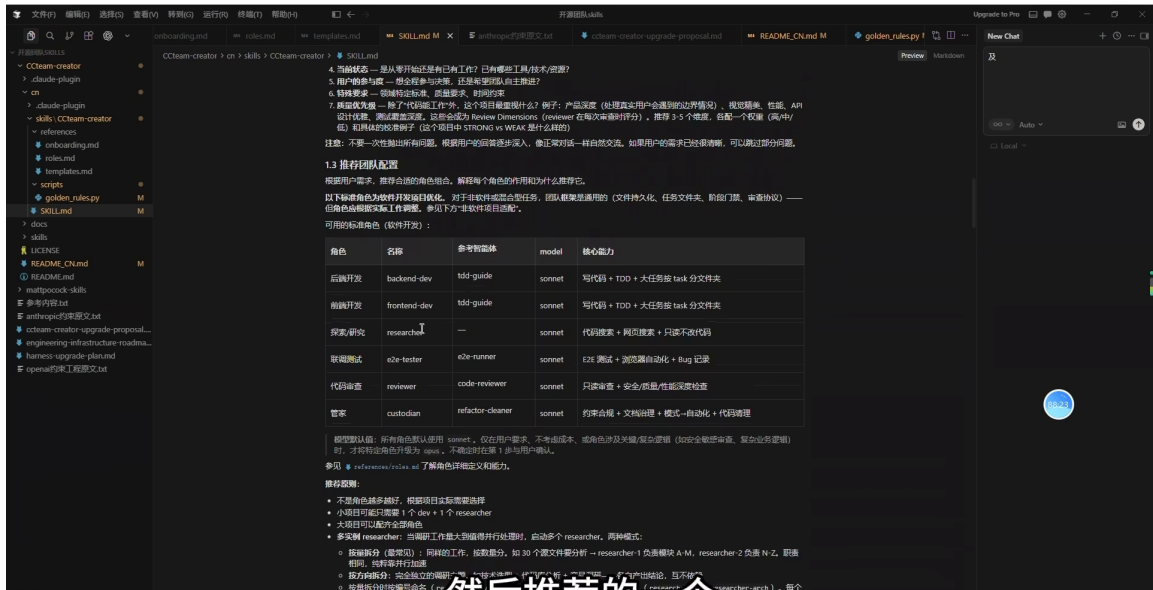


图 10: CCTeamQuarter 角色详细配置表格，展示各子智能体的模型参数与职责定义。

有意思的是，讲师在实践中发现，这套流程在 GPT 系列模型上的表现反而比 Claude Cloud 系列更稳定。这说明 team 工作流的效果不仅取决于 Prompt 设计，也与模型对长指令和角色扮演的遵循能力密切相关。

5.3 文档树与热更新导航地图的架构

如果说角色是 team 的”人”，那么文档系统就是 team 的”记忆”。CCTeamQuarter 的文档架构借鉴了”plan with files”思想，但做了一个关键改进：**按任务维度再拆分一层**。

为什么每个任务都要有自己的三文档？

很多早期的 team 方案只在项目根目录放三个文件 (plan、findings、progress)。当项目变大后，这三个文件会像”1000 页的说明书”一样膨胀，模型读不完、抓不住重点。CCTeamQuarter 的做法是：每个角色下按任务建立文件夹，每个文件夹里再放自己的 plan / findings / progress。上层文档只保留摘要和索引，防止单文件爆炸。

一个典型的 CCTeamQuarter 文档树如下：

```

1 project-docs/
2 |-- leader-plan.md           # Leader 主计划（团队级汇总）
3 |-- architecture-decisions.md # 架构决策记录
4 |-- api-spec.md              # 前后端 API 规范（防止漂移）
5 |-- navigation-map.md        # 热更新导航地图（由管家维护）
6 |-- members/
7     |-- backend/
8         |-- task-001-login/
9             |-- plan.md
10            |-- findings.md
11            |-- progress.md
12     |-- frontend/

```

⁰ 视频画面时间区间：01:06:00–01:10:00。

```

13 |   |-- task-001-login/
14 |       |-- plan.md
15 |       |-- findings.md
16 |       |-- progress.md
17 |-- researcher/
18     |-- task-001-login/
19         |-- plan.md
20         |-- findings.md
21         |-- progress.md

```

Listing 7: CCTeamQuarter 文档树结构示例

导航地图的热更新

讲师反复强调：**不要把导航地图全部塞进 cloud.md**。原因是 cloud.md 属于系统提示的一部分，除非重启对话，否则无法热更新。而项目的进展、新任务的创建、旧任务的归档都是动态变化的。因此，导航地图被设计成一个独立的 navigation-map.md，由管家持续维护。Leader 和各个子智能体可以随时读取这个文件，快速定位到当前需要关注的任务文档。

避免文档孤岛

文档树里的任何一份文件都不应该是“孤岛”。它必须被某个上层索引引用，哪怕只是一句简单的摘要说明。孤立的文档对模型来说几乎等于不存在——因为模型不会无缘无故去搜索一份未被引用的文件。

规范文档：防止漂移的最后一道防线

在文档树中，api-spec.md 和 architecture-decisions.md 属于**规范文档**。API 的端口、路由、变量命名一旦写入，就不能随意更改；如果要改，必须同步更新文档。这样做是为了防止模型在上下文不足时“乱编”新的变量或接口，导致代码漂移。

讲师建议，代码量与文档量的比例尽量不要低于 20%。他自己的项目中文档约占 30%，虽然看起来有点多，但在长周期、多角色的 team 工作流程中，充足的文档是避免上下文腐烂和局部最优的唯一解药。

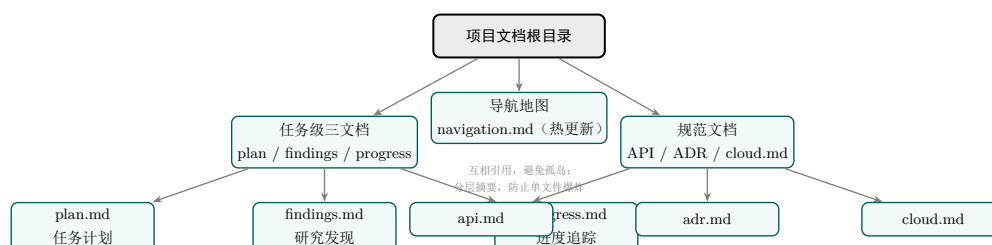


图 11: CCTeamQuarter 文档树结构示意图：任务级三文档、导航地图与规范文档的分层关系。

⁰ 基于视频内容由 TikZ 绘制。

5.4 落地建议：从 Manual 到自动 hook 的演进

CCTeamQuarter 目前的状态是”高度结构化但尚未全自动”。讲师给出的落地路径分为三个阶段：

1. **Manual 阶段**：由用户手动触发 skill，Leader 与用户沟通后再派发任务。此阶段主要用于磨合角色定义和文档规范。
2. **半自动阶段**：加入 CI 检查脚本、自动 review 触发器和测试报告汇总。此时用户只需在关键节点做确认。
3. **全自动阶段**：通过添加 hook 脚本，实现”问题输入 → 团队自动分析/开发/测试/部署 → 结果反馈”的闭环。当人工接受率稳定在 90% 以上时，即可进入此阶段。

CLI 化优先的落地原则

在推行 team workflow 时，依然要坚持前面章节提到的原则：功能验证先做 CLI，再包装前端。CCTeamQuarter 的前后端正反角色虽然会产出 UI，但核心逻辑的测试和调试最好通过命令行完成。只有 CLI 化之后，才能更容易地被 skill、CI 和自动化 hook 调用。

关于 token 成本，讲师纠正了一个常见误解：Team 模式看起来人多嘴杂，似乎很烧钱，但实际上比反复新建子智能体更省。因为 Team 模式内的上下文是可以复用的；而子智能体每次激活都要重新加载上下文，反而会导致更高的 KV Cache 重置开销。

最后，讲师提到目前第三方中转站（API 代理）市场的不透明问题：很多中转站不提供 **KV Cache** 命中率数据，甚至根本不启用 KV Cache 复用。这导致表面上的”低价”在实际 team 长对话中非常昂贵。如果你在 2o 模型上使用这类中转站，月成本可能高达 3000-5000 美元。这一点在评估团队工作流的运营预算时务必注意。

5.5 本章小结

本章围绕 CCTeamQuarter 开源项目，完整展示了从零构建一套 Team workflow 的方法论与落地细节：

- **定位**：CCTeamQuarter 是一个可复用的 Cloud Code Skill，固化了讲师在大型项目中验证的 team 协同流程，复现了 Harness Engineering 约 60%-70% 的核心思想。
- **启动**：通过读取参考文件、检查 plan 文档、扫描代码库、对齐目标、生成 cloud.md 五步完成 team 初始化；已有配置可复用以节省 token。
- **角色**：推荐 Leader + 前后端 + 研究者 + Review + E2E 测试 + 管家六角色；拆分原则是”上下文可分离、沟通成本低”；子智能体与 Leader 之间需要多次对齐需求。
- **文档**：采用”任务级三文档（plan / findings / progress）+ 团队级汇总 + 热更新导航地图”的树状结构；规范文档（API、架构决策）是防止代码漂移的关键防线。
- **演进**：从 Manual 磨合，到加 CI/hook 半自动，最终在 90% 以上接受率时实现全自动；同时警惕不透明中转站带来的 KV Cache 成本陷阱。

到此，我们已经走完了从单智能体瓶颈、上下文原理、Cloud Code 进阶操作、约束工程，再到完整 Team workflow 落地的全部旅程。希望你能在自己的项目中，借助 CCTeamQuarter 的思想，让 AI 团队真正跑起来。

6 总结与延伸

回顾这近 80 分钟的高密度分享，讲师最核心的论点始终没变：**AI 不是魔术师，它是一匹需要缰绳的野马**。大到 OpenAI 的百万行代码项目，小到个人开发者的首个全栈应用，Harness Engineering 的底层逻辑都同样适用——用文档划定边界、用测试固化约束、用可观测性照亮黑盒。

6.1 讲师的收尾建议

在视频的最后，讲师留下了几条可直接执行的落地建议：

1. **先跑通 Manual，再谈全自动**。CCTeamQuarter 目前仍是“高度结构化但需人工触发”的状态。不要一上来就追求 hook 和死循环，先把角色定义、文档规范、CI 检查磨合到 90% 以上的人工接受率，再加自动化。
2. **警惕不透明中转站**。很多第三方 API 代理打着低价的旗号，实则不提供 KV Cache 命中率，甚至不启用缓存复用。在 Team 模式的长对话场景下，这种“低价”可能让你的月账单翻上数倍。
3. **别把导航地图塞进 cloud.md**。项目的进展是动态的，cloud.md 是静态的系统提示。让管家把导航地图维护成独立的 navigation-map.md，才能实现热更新。
4. **文档量不要低于代码量的 20%**。在长周期、多角色的协作中，充足的文档是防止上下文腐烂和局部最优的唯一解药。

6.2 我们的综合提炼

除了讲师的原话，我们还可以从更抽象的视角理解这套方法论：

三层抽象

- **物理层（上下文与缓存）**：KV Cache 和手动压缩是长对话的经济基础，没有这个基础，任何复杂工作流都会因成本或记忆断裂而崩塌。
- **操作层（Cloud Code 技巧）**：模式选择、模型搭配、对话管理和插件可视化，构成了这套工作流的“操作系统”。
- **制度层（Harness + CCTeamQuarter）**：文档规范、CI 约束、角色分工和多层导航地图，构成了让 AI 团队长期稳定运行的“上层建筑”。

6.3 下一步行动

如果你也想在自己的项目中引入类似的 Team 工作流，可以从以下最小可行路径开始：

1. **本周**：读一遍 OpenAI 的 Harness Engineering 原文，并安装 Cloud Hold 插件监控你的上下文消耗。
2. **本月**：在一个小型项目中试用 CCTeamQuarter 的六角色模板，重点观察 plan / findings / progress 三文档是否真的降低了返工率。
3. **本季**：把你的第一套 CI 约束脚本（行数检查、变量名检查、E2E 测试）跑起来，让它成为模型不可逾越的硬边界。